1. (3 Punkte) Gegeben ist das folgende Programm mit vier Kommentaren.

```
a = [3, -1, 4]          # (1) Liste a
s = 0                   # (2) setze s auf 0
# (3) von hinten, weil pop die Indizes der folgenden Elemente verschiebt
for i in range(len(a)-1, -1, -1):
    if a[i] < 0:
        a.pop(i)        # (4) entferne das Element
```

   (a) Welcher Kommentar ist sinnvoll?

   (b) Begründe, warum die anderen Kommentare nicht sinnvoll sind.

> **Lösung:** a) Kommentar (3).
> b) Die Kommentare (1), (2) und (4) wiederholen nur, was im Code steht. Kommentar (3) erklärt, warum die Schleife rückwärts läuft.

2. (3 Punkte) Gegeben ist die folgende Funktion.

```
def endpreis(preis):
    # ab 100 Euro gibt es 10 Prozent Rabatt
    if preis > 50:
        return preis * 0.8
    return preis
```

   (a) Der Kommentar passt nicht zum Code. Nenne die beiden Abweichungen.

   (b) Warum ist ein falscher Kommentar schlimmer als gar kein Kommentar?

> **Lösung:** a) Im Kommentar steht ab 100 Euro, im Code ab mehr als 50 Euro. Im Kommentar stehen 10 Prozent, im Code werden 20 Prozent abgezogen.
> b) Wer den Kommentar liest, verlässt sich darauf und wird in die Irre geführt.

3. (2 Punkte) Ergänze an der markierten Stelle einen Kommentar, der erklärt, warum die Schleife nur bis zur Wurzel von n läuft.

```
def ist_prim(n):
    if n < 2:
        return False
    t = 2
    # Kommentar
    while t * t <= n:
        if n % t == 0:
            return False
        t = t + 1
    return True
```

> **Lösung:**
> `# Zu einem Teiler über der Wurzel gehört immer ein Teiler unter der Wurzel.`

4. (3 Punkte) Schreibe die Funktion so um, dass sie durch gute Namen und eine Konstante ohne Kommentare verständlich ist.

```
def f(a, b):
    # a: Stunden, b: Stundenlohn, 20 Prozent gehen ab
    return a * b * 0.8
```

Lösung:

```
NETTO_ANTEIL = 0.8

def nettolohn(stunden, stundenlohn):
    return stunden * stundenlohn * NETTO_ANTEIL
```

5. (2 Punkte) Die folgenden Namen entsprechen nicht PEP 8. Gib jeweils einen passenden Namen an.
   (a) die Variable `AnzahlSchueler` und die Funktion `BerechneSumme`
   (b) die Klasse `bank_konto` und die Konstante `maxpunkte`

Lösung: a) `anzahl_schueler`, `berechne_summe`
b) `BankKonto`, `MAX_PUNKTE`

6. (3 Punkte) Ergänze einen Docstring, der die Argumente und den Rückgabewert beschreibt und zwei Beispielaufrufe enthält.

```
def zaehle_vokale(s):
    z = 0
    for c in s.lower():
        if c in 'aeiou':
            z += 1
    return z
```

Lösung:

```
def zaehle_vokale(s):
    '''
    s: String
    returns: Anzahl der Vokale a, e, i, o, u in s, auch als Großbuchstaben

    >>> zaehle_vokale('Hallo')
    2

    >>> zaehle_vokale('Otto')
    2
    '''
    z = 0
    for c in s.lower():
        if c in 'aeiou':
            z += 1
    return z
```

7. (3 Punkte) Schreibe einen Docstring zu der folgenden Funktion. Beschreibe auch, was die Funktion mit ihrem Argument macht.

```
def entferne_negative(a):
    for i in range(len(a)-1, -1, -1):
        if a[i] < 0:
            a.pop(i)
```

Lösung:

```
    '''
    a: Liste von Zahlen
    verändert a: alle negativen Zahlen werden entfernt
    returns: None

    >>> b = [1, -2, 3]
    >>> entferne_negative(b)
    >>> b
    [1, 3]
    '''
```

8. (4 Punkte) Ergänze Docstrings für die Klasse und für die Methoden `fahrenheit` und `erhoehen`.

```
class Thermometer:
    def __init__(self, celsius):
        self.celsius = celsius

    def fahrenheit(self):
        return self.celsius * 9 / 5 + 32

    def erhoehen(self, grad):
        self.celsius += grad
```

Lösung:

```
class Thermometer:
    '''Ein Thermometer, das eine Temperatur in Grad Celsius speichert.'''

    def __init__(self, celsius):
        self.celsius = celsius

    def fahrenheit(self):
        '''
        returns: die Temperatur in Grad Fahrenheit
        '''
        return self.celsius * 9 / 5 + 32

    def erhoehen(self, grad):
        '''
        Erhöht die Temperatur.
        grad: Zahl, um die sich die Temperatur in Grad Celsius erhöht
        '''
        self.celsius += grad
```

9. (4 Punkte) Das folgende Programm läuft, verstößt aber mehrfach gegen PEP 8. Schreibe es nach den Regeln von PEP 8 neu.

```
def QuadratSumme( a,b ) :
  S=a*a+b*b
  return S
print(QuadratSumme(3,4))
```

Lösung:

```
def quadrat_summe(a, b):
    s = a * a + b * b
    return s


print(quadrat_summe(3, 4))
```

10. (2 Punkte) Beantworte die folgenden Fragen.
    (a) Beim Testen sollen fünf Zeilen vorübergehend nicht ausgeführt werden. Wie geht das in der Entwicklungsumgebung am schnellsten?
    (b) Wie lässt man sich den Docstring einer Funktion `f` anzeigen?

Lösung: a) Zeilen markieren und auskommentieren, in Thonny mit Strg + 3, in Jupyter mit Strg + /.
b) Mit `help(f)`, in Jupyter auch mit Shift + Tab oder `f?`.

11. (3 Punkte) Beantworte die folgenden Fragen.
    (a) Was ist der Unterschied zwischen einem Kommentar und einem Docstring?
    (b) Was regelt PEP 8?
    (c) Warum sollte man Kommentare auf das Warum beschränken und nicht das Was beschreiben?

Lösung: a) Ein Kommentar richtet sich an die Leser des Codes und wird von Python ignoriert. Ein Docstring beschreibt die Benutzung einer Funktion oder Klasse, Python speichert ihn und `help` zeigt ihn an.
b) PEP 8 ist der Style Guide für Python: Namen und Formatierung des Codes.
c) Was geschieht, steht schon im Code. Der Grund für eine Entscheidung steht dort nicht.